

```
timescale 1ns / 1ps
`default_nettype none
`include "parameters.vh"
```

```
module IF (
    input wire clk,
    input wire rst_n,
    input wire stall,

    output wire [31:0] o_PcD,
    output wire [31:0] o_InstrD,

    // Instruction Memory Interface
    input wire [31:0] i_Inst,
    input wire i_Imem_ack,
    output reg o_Imem_stb,
    output reg [31:0] o_Iaddr,

    // PC Change Control Signals
    input wire [31:0] i_Imm,
    input wire [31:0] i_Result,
    input wire i_Boj,
    input wire i_Jalr
);

// Internal signals and registers
reg [31:0] pc, pc_reg, Instr_reg;

// Stall conditions
wire is_stall = (!i_Imem_ack && ~rst_n) || stall;

// AMO detection from instruction opcode
wire is_amo = (i_Inst[6:0] == `AMO);

// Next PC logic
wire [31:0] pc_next = i_Boj      ? i_Imm :
                      i_Jalr     ? (i_Result & ~32'b1) :
                                pc + 32'd4;

assign o_PcD      = pc_reg;
assign o_InstrD   = Instr_reg;

// PC update logic
always @(posedge clk) begin
    if (~rst_n) begin
        pc <= `PC_RESET;
    end
end
```

```

        end else if (!is_stall) begin
            pc <= pc_next;
        end
    end

// Instruction fetch pipeline registers
always @(posedge clk) begin
    if (~rst_n || i_Boj) begin
        pc_reg <= `PC_RESET;
        Instr_reg <= `NOP;
    end else if (!is_stall) begin
        pc_reg <= pc;
        Instr_reg <= i_Inst;
    end
end

// Instruction memory interface
always @(*) begin
    if (~rst_n) begin
        o_Iaddr = 32'd0;
        o_Imem_stb = 1'b0;
    end else begin
        o_Iaddr = pc;
        o_Imem_stb = 1'b1;
    end
end

// Simulation logging for debugging
`ifdef SIM
integer fd;
always @(o_PcD, Instr_reg) begin
    #2;
    if (rst_n && o_Imem_stb) begin
        fd = $fopen("IF_log.csv", "ab+");
        $fwrite(fd, "%h,%h\n", o_PcD, Instr_reg);
        $fclose(fd);
    end
end
`endif
endmodule

```